

*** Difference between throw and throws :

throw [THROWING THE EXCEPTION OBJECT EXPLICITLY.]

throw keyword is used to explicitly create and throw an exception object.

It can throw only one exception at a time inside the method body, the control immediately transfers to the nearest catch block.

We can only throw the exception which are sub type of Throwable (Then only we can say It is Throwable Object)

```
        throw new ArithmeticException();
        throw new LowBalanceException();
```

Note : LowBalanceException class must be of type Throwable

throws :-

throws keyword describes that the method might throw an Exception, It also might not. (Exception encountered at runtime only)

It is used only at the end of a method declaration to indicate what exceptions it supports OR what type of Exception it might throw at runtime.

It is used to skip from the current situation so now the exception will be propagated to the caller method OR JVM for handling purpose.

It is used to work with Checked Exception as well as we can add more exception at the method declaration must be separated by comma.

```
Example :
public void m1() throws IOException, ClassNotFoundException
{
}
```

How to develop the Custom Exception : [Industry Requirements]

As a developer we can develop user-defined checked OR user-defined unchecked exception.

If we want to develop checked exception then our user-defined class must extends from java.lang.Exception, on the other hand if we want to develop un-checked exception then our user-defined class must extends from java.lang.RuntimeException.

In the user-defined exception class we should write No argument constructor(in case if we don't want to pass any error message) and we should write parameterized constructor with String errorMessage as a parameter (in case if we want to pass any error message) with super keyword.

In order to throw the exception object explicitly we should use throw keyword as well as our user-defined class must be an instance of Throwable type.[Must be a Throwable Object]

```
package com.ravi.custom_exception;

import java.util.Scanner;

class InvalidAgeException extends Exception
{
    //Version compatibility
    private static final long serialVersionUID = 1L;

    public InvalidAgeException()
    {
    }

    public InvalidAgeException(String errorMessage)
    {
        super(errorMessage);
    }
}

public class CustomCheckedException
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        try(sc)
        {
            IO.print("Enter your Age :");
            int age = sc.nextInt();
            validateAge(age);
        }
        catch(InvalidAgeException e)
        {
            IO.println(e.getMessage());
        }
    }

    public static void validateAge(int age) throws InvalidAgeException
    {
        if(age < 18)
        {
            throw new InvalidAgeException("Age is invalid");
        }
        else
        {
            IO.println("You are eligible for vote");
        }
    }
}
```

```
package com.ravi.custom_exception;

import java.util.InputMismatchException;
import java.util.Scanner;

class GreaterMarksException extends RuntimeException
{
    private static final long serialVersionUID = 1L;

    public GreaterMarksException()
    {
    }

    public GreaterMarksException(String errorMessage)
    {
        super(errorMessage);
    }
}

public class CustomUncheckedException
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        try(sc)
        {
            IO.print("Enter your marks :");
            int marks = sc.nextInt();
            validateMarks(marks);
        }
        catch(GreaterMarksException e)
        {
            IO.println(e);
        }
        catch(InputMismatchException e)
        {
            IO.println("Input is not is a proper format");
        }
        catch(Exception e)
        {
            IO.println("General Exception");
        }
    }

    public static void validateMarks(int marks)
    {
        if(marks > 100)
        {
            throw new GreaterMarksException("Invalid marks ");
        }
        else
        {
            IO.println("Your Marks is :"+marks);
        }
    }
}
```

protected native Object clone() throws CloneNotSupportedException :

* predefined non static method of Object class.

* It is used to create a **duplicate copy of an object** (cloned object) in a different memory location. [If we modify original object then cloned object will remain un-changed]

* It is protected method so, we need to override this method in the BLC class, It throws a checked exception i.e. CloneNotSupportedException.

* IF WE WANT TO PERFORM CLONING OPERATION ON ANY CLASS OBJECT THEN THAT PARTICULAR CLASS MUST IMPLEMENTS FROM java.lang.Cloneable marker interface which provides an additional information to JVM that object is a cloneable object otherwise we will get a runtime exception CloneNotSupportedException

```
package com.ravi.clone;

class Product implements Cloneable //Providing additional information to JVM regarding the Object
{
    private int id;
    private String name;

    public Product(int id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public String toString()
    {
        return "Product [id=" + id + ", name=" + name + "]";
    }

    public void setId(int id)
    {
        this.id = id;
    }

    public void setName(String name)
    {
        this.name = name;
    }

    //Method is protected so need to override
    @Override
    public Object clone() throws CloneNotSupportedException
    {
        return super.clone();
    }
}

public class CloneOperation
{
    public static void main(String[] args) throws CloneNotSupportedException
    {
        Product p1 = new Product(1, "Laptop");
        Product p2 = (Product) p1.clone();

        IO.println("Printing both the objects :");
        IO.println(p1+" : "+p2);

        IO.println("Modify the original object :");
        p1.setId(2); p1.setName("HP Laptop");

        IO.println("After modification Printing both the objects :");
        IO.println(p1+" : "+p2);
    }
}
```